A HETEROGENEOUS RELATIONAL ST-GNN FOR NBA TRAJECTORY PREDICTION

Name 1
email1

Name 2
email2

Julien Stalhandske
julien.stalhandske@epfl.ch

Missipsa M'Hamed Annane
missipsa.annane@epfl.ch

ABSTRACT

We predict the next $H{=}12$ frames of an NBA play given a $C{=}8$-frame context of the (x, y) positions of 10 players and the ball. The 11 entities of three distinct types interact through physically meaningful relations (teammate, opponent, possession), so the problem is a natural fit for a heterogeneous graph neural network. We build a per-timestep relational graph with five learned relation types, push it through a stack of *HeteroSTGCN* cells (RGAT $\rightarrow$ GRU $\rightarrow$ LayerNorm), and roll the predicted positions back into the input autoregressively. A court-symmetric data augmentation – axis flips, team-label swap, within-team player permutation – multiplies the effective training set by $\sim 10^5$ because the architecture is equivariant under all four transformations. Averaged over five seeds, the model attains a validation MSE of 3.25 ± 0.16 ft^2; the per-horizon ADE grows almost linearly to 3.49 ft at step 12, with the ball contributing $2.6\times$ the ADE of either team.

1 INTRODUCTION

Predicting the short-horizon motion of all entities on a basketball court is challenging because the interactions between players and the ball are heterogeneous. The EPFL EE-452 Kaggle task formalises this as a sequence-to-sequence problem: given the 11 entity positions over $C{=}8$ context frames, predict the next $H{=}12$ frames.

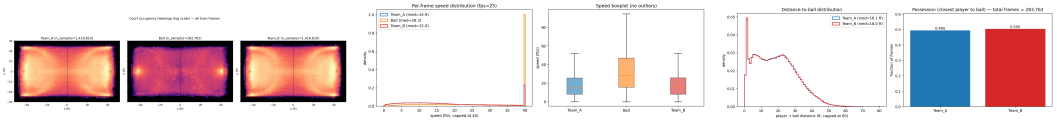
Two properties of the task drive our modelling choices. First, the $N{=}11$ entities are not exchangeable: three node types (Team_A, Team_B, Ball) and the relation between two entities depends on *which* two they are – a heterogeneous graph with multiple relation types is the natural backbone. Second, the court is symmetric about both axes, team labels are arbitrary, and the five players on a team are interchangeable. Any equivariant model can therefore be trained on an effective dataset $2^3 \cdot (5!)^2 {=} 115{,}200$ times larger than the raw files, which we exploit with on-the-fly augmentation.

Our main empirical findings are: (a) train and validation losses overlap inside their 95% confidence bands across five seeds, so the architecture is not overfit at 50 epochs; (b) error grows almost linearly with prediction horizon – the expected signature of autoregressive rollout error accumulation; and (c) the ball dominates the error budget at $2.6\times$ the ADE of either team, suggesting a ball-specific specialisation is the highest-leverage follow-up.

2 METHOD

2.1 DATASET AND TASK

The training set contains $4{,}968$ sequences of length $T \in [20, 225]$ (median 30); the test set has $1{,}243$ files of exactly 8 frames. At each frame the eleven entities are stored as $(x, y, \text{entity\_id}, \text{team\_id})$ with $\text{team\_id} \in \{-1, 0, +1\}$ marking Team_A, the Ball, and Team_B. The spread of sequence lengths yields $189{,}370$ usable $(C{+}H)$-windows, 38 per sequence on average – enough that the model is not data-starved. Figure 1 shows the basic dataset descriptors. The court-occupancy heatmap is mirror-symmetric about both axes for the two teams and the ball concentrates sharply at the two baskets at $x \approx \pm 43$ ft, empirical confirmation that the symmetries we want to encode are present in the data. Possession (closest player to ball) is balanced to $49.5\% \,/\, 50.5\%$ across $283{,}762$ frames.



(a) Court occupancy (log). (b) Speed by entity type. (c) Player–ball distance.

Figure 1: Train-set diagnostics. Occupancy is mirror-symmetric, the ball moves about twice as fast as the players (28.3 vs. 15.0 ft/s median), and the player–ball distance has a sharp ball-handler mode at $\sim$2 ft.

2.2 HETEROGENEOUS GRAPH

Each batch sample induces a complete directed graph on $N{=}11$ nodes with $N^2{=}121$ edges, but each edge carries one of five *relation types* determined by the static team labels at $t{=}0$: $\{0{:}\text{teammate}, 1{:}\text{opponent}, 2{:}\text{player}{\to}\text{ball}, 3{:}\text{ball}{\to}\text{player}, 4{:}\text{self-loop}\}$. This yields $40{+}50{+}10{+}10{+}11$ edges per sample (Figure 2, right). The relation matrix encodes node-type partitioning explicitly and is the only piece of structural information the network needs at construction time. Edges are rebuilt once per batch (since relations are static in time) while per-edge geometric features $[\Delta x, \Delta y, \|\Delta\|]$ are recomputed every step from the rolled-out positions.

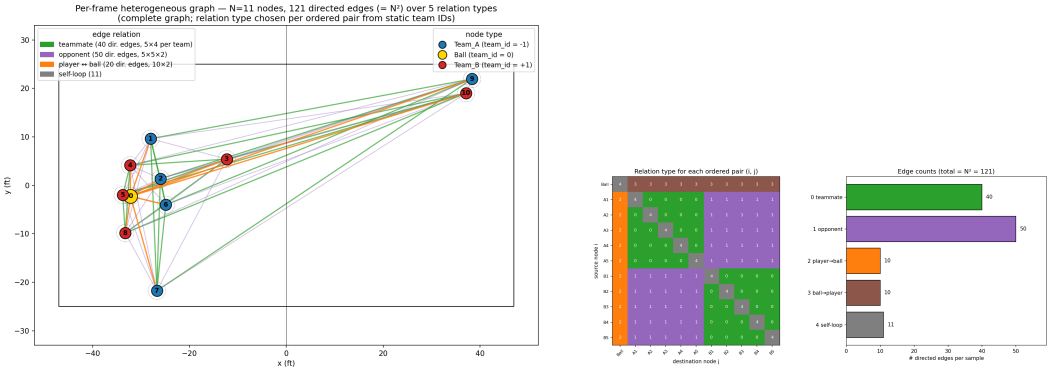


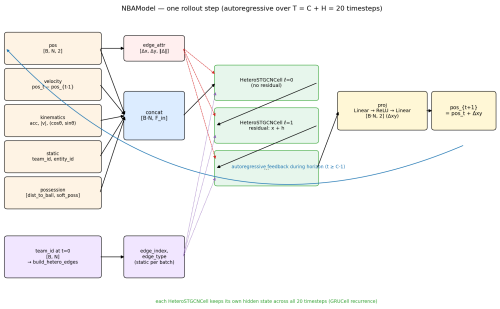
Figure 2: Heterogeneous graph construction. *Left*: the five relation types drawn on a real frame. *Right*: the relation type chosen for each ordered pair (i, j) and the per-relation edge count.

2.3 MODEL ARCHITECTURE

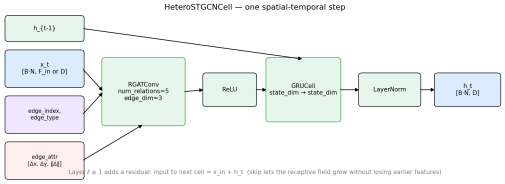
The backbone is a spatial-temporal heterogeneous graph RNN. At each of the $T{=}20$ timesteps the model builds a per-node feature vector (Figure 3c) consisting of the current position, finite-difference velocity, kinematics (acceleration, speed, heading $(\cos\theta, \sin\theta)$), the static $(\text{entity\_id}, \text{team\_id})$ fields, and a possession signal $(d_{\text{ball}}, \text{softmax}(-d/\tau))$ with $\tau{=}5$ ft (Figure 3d). The softmin assigns a peaked probability to the ball-handler and is masked to zero at the ball itself.

The features feed a stack of **HeteroSTGCN** cells (Figure 3b); each cell is an RGAT convolution (Busbridge et al., 2019) (with the five relation types and per-edge geometric features as edge attributes) followed by `ReLU`, a `GRUCell` (Cho et al., 2014) that keeps a per-layer hidden state across

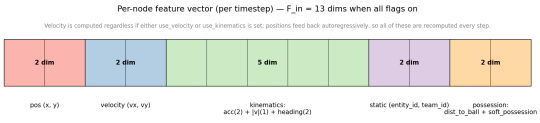
all twenty timesteps, and `LayerNorm`. From layer $\ell \geq 1$ the input to the next cell adds a residual, so the receptive field on the heterogeneous graph grows by one hop per layer without erasing earlier features. A two-layer MLP projects the final hidden state to a 2-D coordinate delta Δ_{xy} and the rollout proceeds via $p_{t+1}{=}p_t + \Delta_{xy}$. During the context window the input position is ground truth; during the horizon it is the model's own previous prediction (Figure 3a).



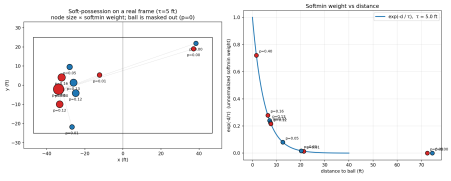
(a) One rollout step.



(b) HeteroSTGCN cell internals.



(c) Per-node feature vector ($F_{\text{in}}{=}13$ with all flags on).



(d) Soft-possession ($\tau{=}5$ ft).

Figure 3: Model overview. The graph and the cells are shared across all $T{=}20$ timesteps; only the per-step inputs change between context and horizon.

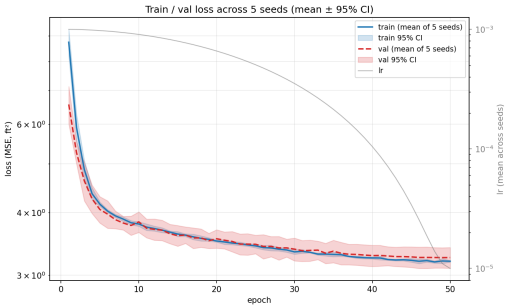
2.4 Training

We minimise MSE over the full $[H, B{\cdot}N, 2]$ prediction tensor with Adam ($\eta{=}10^{-3}$, weight decay $5{\times}10^{-4}$) and `CosineAnnealingLR` ($\eta_{\min}{=}10^{-5}$, $T_{\max}{=}$epochs). Gradients are clipped at $\|\cdot\|_2{=}10$. The model interior runs in normalized units, but all reported metrics are in feet.

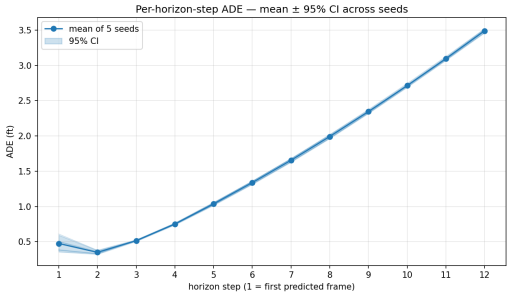
A central design decision is the on-the-fly augmentation: every fetched (C, H) window is independently x-flipped, y-flipped, team-swapped, and within-team permuted with probability $1/2$ each. The court is symmetric about both axes, and the heterogeneous edge construction depends only on team-*equality* (not on which team is $+1$ vs. -1), so all four transformations preserve the graph topology and the loss – they are true symmetries of the model.

3 Results

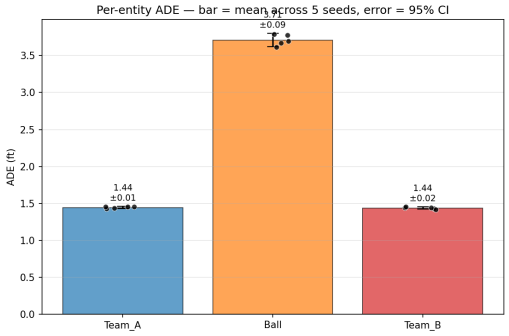
Experimental setup. The best configuration uses $C{=}8$, $H{=}12$, state_dim$=128$, a single HeteroSTGCN layer, batch size 64, five $(\text{seq\_idx}, \text{start})$ windows per training sequence per epoch, and 50 epochs with augmentation. We train five independent models, varying only the master seed (which controls weight initialisation, train/val split, sampler shuffling, and CUDA RNG). All numbers below are mean $\pm$ 95% Student-t confidence interval over the five seeds.



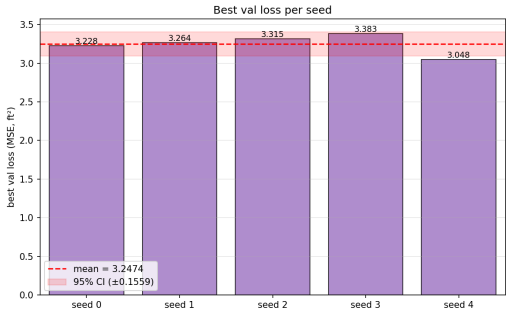
(a) Train/val loss across 5 seeds.



(b) Per-horizon-step validation ADE.



(c) Per-entity validation ADE.



(d) Best val loss per seed.

Figure 4: Multi-seed aggregate results (n=5, 95% CI). Bands and error bars are tight on every metric except step-1 ADE, where seeds disagree more.

Training stability. Final-epoch training loss is 3.20 ± 0.01 and validation 3.25 ± 0.16 ft^2; the best-checkpoint validation loss is 3.25 ft^2 (range $[3.05, 3.38]$). Train and validation curves overlap inside their bands throughout training (Figure 4a), so the model is not over-fit at 50 epochs even without explicit early stopping.

Horizon scaling. Per-horizon ADE grows almost linearly from 0.35 ft at step 2 to 3.49 ft at step 12 (Figure 4b). The slope (~ 0.30 ft/frame, ~ 7.5 ft/s at the assumed 25 fps tracking rate) is roughly half the median player speed. This is the expected signature of autoregressive rollout: when the input at step $t{+}1$ is the model's own output at step t, small per-step errors accumulate.

Symmetry between teams. Per-entity ADE (Figure 4c) reads 1.44 ± 0.01 ft for Team_A, 1.44 ± 0.02 ft for Team_B, and 3.71 ± 0.09 ft for the Ball. The two team errors agree to within 0.01 ft – direct empirical evidence that the augmentation makes the learned model team-equivariant in practice.

The ball dominates the error budget. Although there is only one ball against ten players, its ADE is $2.6\times$ that of either team – the unweighted MSE loss therefore puts only $1/11$th of its mass on the ball even though the ball drives most of the residual error.

4 Conclusion

A relational graph attention backbone wrapped in a GRU recurrence is a strong, low-variance baseline for short-horizon NBA trajectory prediction, reaching 3.25 ± 0.16 ft^2 validation MSE across five seeds with no early stopping. The five relation types let one set of parameters serve very different physical interactions, and the court-symmetric augmentation – valid only because relations depend on team-*equality* – multiplies the effective dataset by $\sim 10^5$; the two-decimal-place agreement between Team_A and Team_B ADEs across five seeds confirms equivariance in practice. The remaining gap is the ball-vs-player asymmetry: a ball-weighted loss or an entity-specific output head (already supported by `use_ball_head`) are the most promising follow-ups.

STATEMENT OF CONTRIBUTIONS

We hereby state that all the work presented in this report is that of the authors.

REFERENCES

Dan Busbridge, Dane Sherburn, Pietro Cavallo, and Nils Y. Hammerla. Relational graph attention networks. *arXiv preprint arXiv:1904.05811*, 2019.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Empirical Methods in Natural Language Processing (EMNLP)*, 2014.